軟體工程 (Software Engineering)

第五課：軟體設計、架構與物件導向原則 (Software Design & Architecture)

從通用設計基石、架構推導塑模到 SOLID 匠藝

薛念林 教授
資訊工程學系
逢甲大學

第五章：課程藍圖與核心主題

第一部分：設計基礎與架構模式

- **5.1 軟體設計導論與全景：** 從需求到實作藍圖、設計為何決定成敗、四大設計層次與學習路徑。
- **5.2 通用軟體設計核心原則：** 模組化、抽象化、資訊隱藏、關注點分離、內聚力與耦合度的層次光譜。
- **5.3 經典軟體架構模式與品質權衡：** 六大模式全景總覽、單體 vs. 微服務、非功能品質屬性與 ATAM 權衡。
- **5.4 軟體架構推導與塑模表現：** 超越 Pattern：ASR 驅動架構、UML 常見陷阱、4+1 視角、C4 Model 與 ADR。

第二部分：詳細設計、AI 賦能與統整

- **5.5 SOLID 物件導向設計原則：** SRP、OCP、LSP、ISP 與 DIP 搭配實戰重構範例。
- **5.6 AI 在架構與軟體設計中的賦能與盲點：** 副駕駛典範、4 大落地應用與人機協同防護。
- **5.7 核心概念統整與參考文獻：** 多維度總結評量、經典文獻與研討交流。

5.1 軟體設計導論與全景

> *"構建軟體設計有兩種方法：一種是讓它簡單到明顯沒有缺陷，另一種是讓它複雜到沒有明顯的缺陷。"*
> *— C. A. R. Hoare (圖靈獎得主)*

什麼是軟體設計？ (What is Software Design?)

- **從需求 (What) 到實作 (How) 的轉化過程：**
  - **需求工程 (Requirements)**：定義利益關係人的痛點與功能期待（*建造什麼*）。
  - **軟體設計 (Software Design)**：構思結構分解、組件邊界、資料模型與通訊協定（*如何建造*）。
- **軟體設計的核心任務：**
  - **建立工程藍圖 (Blueprinting)**：產出架構模型、介面合約與規範，作為工程團隊實作的依據。
  - **進行權衡決策 (Trade-off Decision Making)**：在互斥的非功能需求間取得平衡（如效能 vs. 安全性 vs. 開發成本）。
- **「設計」與「寫程式 (Coding)」的本質區別：**
  - 寫程式是用特定程式語言語法實現演算法邏輯。
  - 設計則是決定**模組如何切割、狀態放在哪裡、組件之間如何解耦**以因應未來的變更。

為什麼軟體設計至關重要？ (Why Design Matters?)

- **1. 杜絕「憑感覺寫代碼 (Vibe Coding)」與結構腐敗：**
  - 缺乏設計的程式碼會迅速誘發 Robert C. Martin 指出的「不良設計四大病徵」：
    - **僵直性 (Rigidity)**：改動一處引發數十個檔案骨牌式連鎖修改。
    - **脆弱性 (Fragility)**：修復模組 A 的錯誤，莫名其妙導致毫無關聯的模組 B 崩潰。
    - **不可移植性 (Immobility)**：組件糾纏不清，無法被抽取復用於其他系統。
- **2. 品質屬性是「設計」出來的，不是「測試」出來的：**
  - 可擴展性、容錯力、低延遲與安全性必須從第一天納入架構骨幹，無法靠日後補寫單元測試補救。
- **3. Barry Boehm 缺陷修復成本曲線 (Cost of Change)：**
  - 在**需求與設計階段**修復瑕疵只需 **1 美元**；若拖到**正式上線維護階段**，修復代價飆升高達 **50 到 100 倍**！
- **4. 建立工程團隊的共通語言與平行開發合約：**
  - 清晰的組件切分與介面規範，讓數十名工程師能並行協作而不互相卡關。

軟體設計包含什麼？多層次設計光譜 (Design Spectrum)

- **軟體設計是一套由巨觀至微觀的完整工程光譜：**
  - 🏛️ **1. 架構設計 (Architectural Design / 巨觀結構)：**
    - 系統整體骨架、主要子系統切分、分散式通訊機制與全域架構模式（如分層、MVC、微服務）。
  - 🔌 **2. 介面設計 (Interface Design / 合約與邊界)：**
    - 公開 API 規範、通訊協定、組件間互動合約與資訊隱藏邊界（封裝內部細節）。
  - 🧩 **3. 組件與詳細物件導向設計 (Detailed OO Design / 微觀實作)：**
    - 類別內部結構、演算法、資料結構、SOLID 設計原則與 GoF 設計模式。
  - 💾 **4. 資料與資料庫設計 (Data & Persistence Design / 狀態儲存)：**
    - 概念實體模型、關聯式綱要 (ERD)、NoSQL 文件模型、快取策略與資料一致性模型。
- **使用者介面設計 (UI/UX Design)：**
  - 聚焦於人機互動體驗、認知人因與易用性啟發原則——將於**第六章**深入剖析。

本章學習路徑：為什麼通用設計原則必須先行？

- **教學編排的根本思維：原則先行 vs. 架構先行**
  - 若一開始就背誦架構模式，模式往往淪為孤立、死記的框線圖。
  - 當你徹底理解**關注點分離**，你便能自然洞悉分層架構 (Layered) 的存在必要；
  - 當你深刻掌握**低耦合度**，你便能透徹理解 MVC 為何堅持將 Model 與 View 隔離；
  - 當你深悟**高凝聚度**，你便能明白微服務 (Microservices) 為何必須嚴格對齊領域邊界。
- **步步為營的完整學習軌跡：**
  - **第一步：通用設計原則 (5.2)**：核心基石、內聚力與耦合度的詳細層次劃分。
  - **第二步：經典架構模式與權衡 (5.3)**：六大架構樣式、單體 vs. 微服務技術對決與 ATAM。
  - **第三步：架構推導與塑模 (5.4)**：超越 Pattern：ASR 驅動、UML 陷阱、4+1 視圖、C4 Model 與 ADR。
  - **第四步：微觀詳細設計 (5.5)**：類別層級的 SOLID 物件導向工程匠藝。
  - **第五步：現代實踐與 AI 賦能 (5.6 & 5.7)**：AI 副駕駛工作流、防範盲點與全景統整。

5.2 通用軟體設計核心原則

> *"優良的設計極易修改；拙劣的設計在每一步都抗拒改變。"*
> *— Dave Thomas (The Pragmatic Programmer)*

通用原則：貫穿全尺度的「分形 (Fractal)」特性

- **軟體設計具有分形特性 (Fractal Nature)：**
  - 同一套核心法則，在不同層級的放大鏡下展現出完全一致的規律：
    - **系統巨觀層級 (System Scale)**：微服務架構將業務領域拆解為獨立解耦、高凝聚的自治服務。
    - **模組套件層級 (Module Scale)**：Package 與函式庫將特定職責封裝於公開 API 邊界之後。
    - **類別微觀層級 (Class Scale)**：單一類別嚴格封裝狀態，只保留單一的改變理由 (SRP)。
- **原則的本質是為了克服人類心智的極限：**
  - 電腦記憶體有幾十 GB，但人類的「工作記憶 (Working Memory)」只有 7 ± 2 個區塊 (Miller's Law)。
  - 設計原則是工程心智工具，旨在確保大型軟體能被人類大腦理解、維護與擴充。

經典設計準則：模組化、抽象化與關注點分離

- **1. 抽象化 (Abstraction)：**
  - 將低階實作細節隱藏在清晰高階概念之後，讓工程師能在不被雜訊干擾的狀態下推演系統行為。
- **2. 封裝與資訊隱藏 (Encapsulation & Information Hiding)：**
  - *David Parnas 原則：* 將資料與行為綑綁於模組內部，嚴格隱藏私有變數，防止外部程式產生非法副作用。
- **3. 關注點分離 (Separation of Concerns, SoC)：**
  - 將系統切分為互不重疊的功能面向（例如：UI 呈現邏輯絕不混雜 SQL 查詢語法）。
- **4. 模組化 (Modularity)：**
  - 將大型軟體劃分為高內聚、可獨立測試與替換的積木單元。

至高黃金法則：高凝聚度與低耦合度

- **凝聚度 (Cohesion)：**
  - 衡量單一類別或模組內部所有責任與成員彼此關聯的緊密程度。
  - **工程目標： 高凝聚度 (High Cohesion)**（一個模組只專注做好一件事）。
- **耦合度 (Coupling)：**
  - 衡量不同模組之間直接相依與牽扯的強烈程度。
  - **工程目標： 低耦合度 (Low Coupling)**（模組間透過抽象合約互動，對彼此實作細節完全無知）。
- **軟體工程至高黃金法則：**

> *「極大化凝聚度，極小化耦合度！」*
> *高凝聚確保模組專注好懂；低耦合確保改動不會引發骨牌式連鎖崩潰。*

凝聚度與耦合度：不良設計 vs. 優良設計對比

```
// ❌ 劣質設計：低凝聚度 + 高耦合度（臭名昭著的「上帝類別 God Class」）
class OrderManager {
    public void processOrder() {
        // 直接硬編碼寫死 MySQL 資料庫查詢 SQL
        // 計算複雜的 VIP 客戶打折數學公式
        // 手工拼接顧客發票的 HTML 字串
        // 打開底層 TCP/SMTP socket 連線直接發送 Email
    }
}
```

```
// ✅ 優良設計：高凝聚度 + 低耦合度（職責分明、依賴抽象介面）
class OrderProcessor {
    private final DiscountCalculator discountCalculator;
    private final OrderRepository orderRepository;
    private final NotificationService notificationService;

    public OrderProcessor(DiscountCalculator dc, OrderRepository repo, NotificationService ns) {
        this.discountCalculator = dc;
        this.orderRepository = repo;
        this.notificationService = ns;
```

內聚力的層次與類型：從弱到強 (Weak to Strong)

- **內聚力階梯譜系 (Larry Constantine & Edward Yourdon)：**
  - 衡量「單一模組 / 類別內部各元素彼此關聯與聚焦的緊密程度」：

層次	內聚力類型	核心特徵與運作機制	實務範例	品質評估
7	**功能性內聚 (Functional)**	模組內部所有元素只為完成單一、明確且具體的運算或業務功能	`calculateTax()`, `sin(x)`	**最高境界 (最理想)**
6	**順序性內聚 (Sequential)**	前一個操作的輸出，直接作為下一個操作的輸入（如生產線）	`readData() → parseJSON() → validate()`	**強烈推薦**
5	**溝通性內聚 (Communicational)**	各項操作共用相同的輸入資料，或產出相同的輸出資料	`updateCustomerAddress()` 兼 `printLabel()`	**可接受**
4	**過程性內聚 (Procedural)**	各步驟僅因「特定執行先後順序」而被綁在一起，處理不同資料	先 `validateOrder()`，再 `auditLog()`	**普通 (中度)**
3	**時間性內聚 (Temporal)**	元素僅因「在同一時間點執行」而被包在一起	`systemStartup()`, `loadConfigAndConnect()`	**偏弱 (應重構)**
2	**邏輯性內聚 (Logical)**	模組包攬同類任務，由傳入的控制 Flag 決定具體執行哪一段分支	`printReport(int format)` (內部 switch CSV/PDF)	**不良 (違反 OCP)**
1	**偶然性內聚**	元素之間毫無邏輯關聯，僅是隨機雜	`CommonUtils.java`	**最差 (務必**

耦合度的層次與類型：從緊到鬆 (Tight to Loose)

- **耦合度階梯譜系 (模組之間的相互依賴程度)：**
  - 衡量「模組之間呼叫、資料傳遞與知識共享的牽扯深度」：

層次	耦合度類型	依賴機制與破壞風險	實務範例	品質評估
1	**內容耦合 (Content)**	一個模組直接竄改或存取另一個模組的私有內部記憶體或邏輯	繞過封裝直接修改他者 private 欄位	**最差 (極度危險)**
2	**共用耦合 (Common)**	多個模組共同讀寫全域變數或全域共用記憶體狀態	多執行緒下共用未加保護的全域狀態 flag	**高風險 (難追蹤)**
3	**控制耦合 (Control)**	一個模組透過傳遞控制旗標 (Flag) 來支配另一個模組的內部流程	呼叫時傳入 `mode = 3` 操控被呼叫者行為	**中高風險**
4	**標記耦合 (Stamp)**	模組傳遞整包複合資料結構，但被呼叫者僅需其中一小部分	函式只需 `zipCode`，卻傳入整包 `Customer`	**中度 (可改善)**
5	**資料耦合 (Data)**	模組間僅透過簡單、獨立且必要的純參數進行通訊	`calculateArea(width, height)`	**函式呼叫最佳**
6	**訊息/事件耦合 (Message)**	模組間透過非同步訊息/事件通訊，彼此完全不知曉對方存在	EventBus：發布 `OrderPlacedEvent`	**完全解耦 (最理想)**

觀念檢核測驗 1 (CCQ 1)

在軟體系統設計中，將**高凝聚度 (High Cohesion)** 與 **低耦合度 (Low Coupling)** 相互結合，其最主要的實質工程效益為何？

- **A.** 透過將所有函式內聯至單一二進位執行檔中，大幅極大化 CPU 原始運算效能。
- **B.** 產出高度模組化的架構，使單一類別的內部修改對其他模組產生最小的連鎖連帶影響。
- **C.** 強制規定將系統所有共用資料結構集中存放於單一全域變數空間中統一管理。
- **D.** 徹底免除專案撰寫自動化整合測試與回歸測試套件的工程負擔。



5.3 經典軟體架構模式與品質權衡

> *「軟體架構沒有完美的解法，只有持續的權衡取捨。」*
> *— Thomas Sowell*

什麼是軟體架構？ (What is Software Architecture?)

- **正式定義 (IEEE 1471 / ISO 42010)：**

> *軟體架構是系統的根本性組織，體現在其組件、組件彼此與環境之間的相互關係，以及指導其設計與演進的原則中。*

- **架構的「橋樑角色 (The Bridge Role)」：**
  - **需求 (What)**：利益關係人的期望、使用者故事與限制條件。
  - **實作 (How)**：具體程式碼、資料表綱要與測試案例。
  - **架構 (橋樑)**：將混亂的人類需求轉化為結構化的子系統、服務邊界與部屬拓撲。
- **微觀架構 vs. 巨觀架構 (In the Small vs. In the Large)：**
  - **微觀架構 (In the Small)**：單一應用程式內部的模組組織與類別劃分。
  - **巨觀架構 (In the Large)**：企業級分散式系統、雲端叢集、事件匯流排與異質資料庫拓撲。

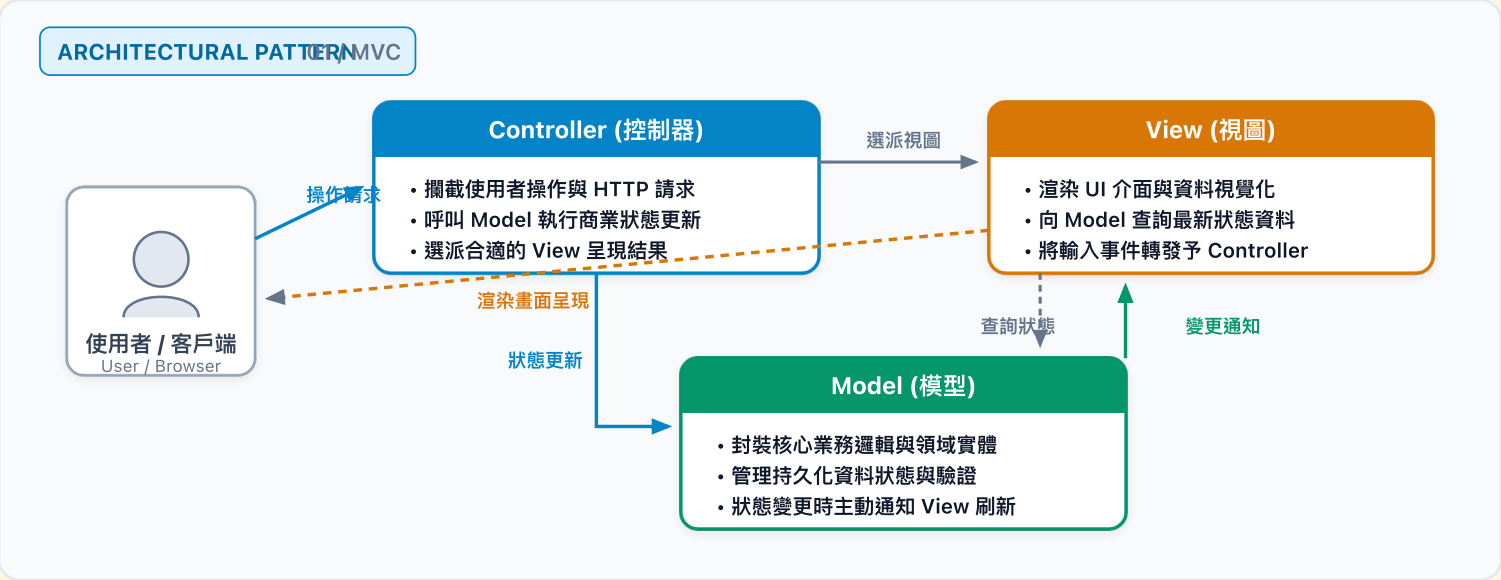
什麼是軟體架構模式？ (Architectural Patterns)

- **架構模式的本質：**
  - 架構模式是在給定軟體工程情境下，針對**反覆出現的結構性設計問題，所提出經過實戰檢驗的通用可復用解法**。
  - 設計模式 (Design Patterns) 作用於微觀類別層次；架構模式則決定**整個系統的巨觀宏觀結構拓撲**。
- **架構模式的三大核心要素：**
  - **1. 情境 (Context)**：特定反覆出現的軟體系統開發環境與工程挑戰。
  - **2. 問題與作用力 (Problem & Forces)**：相互抗衡的需求拉扯（如：資料解耦 vs. 即時性延遲）。
  - **3. 解法與代價 (Solution & Consequences)**：組件劃分、通訊約束與必須接受的缺點妥協。
- **為什麼架構模式至關重要：**
  - 為大型工程團隊建立共通詞彙，避免重複犯下歷史上的經典架構錯誤。

六大經典架構模式全景對照總表

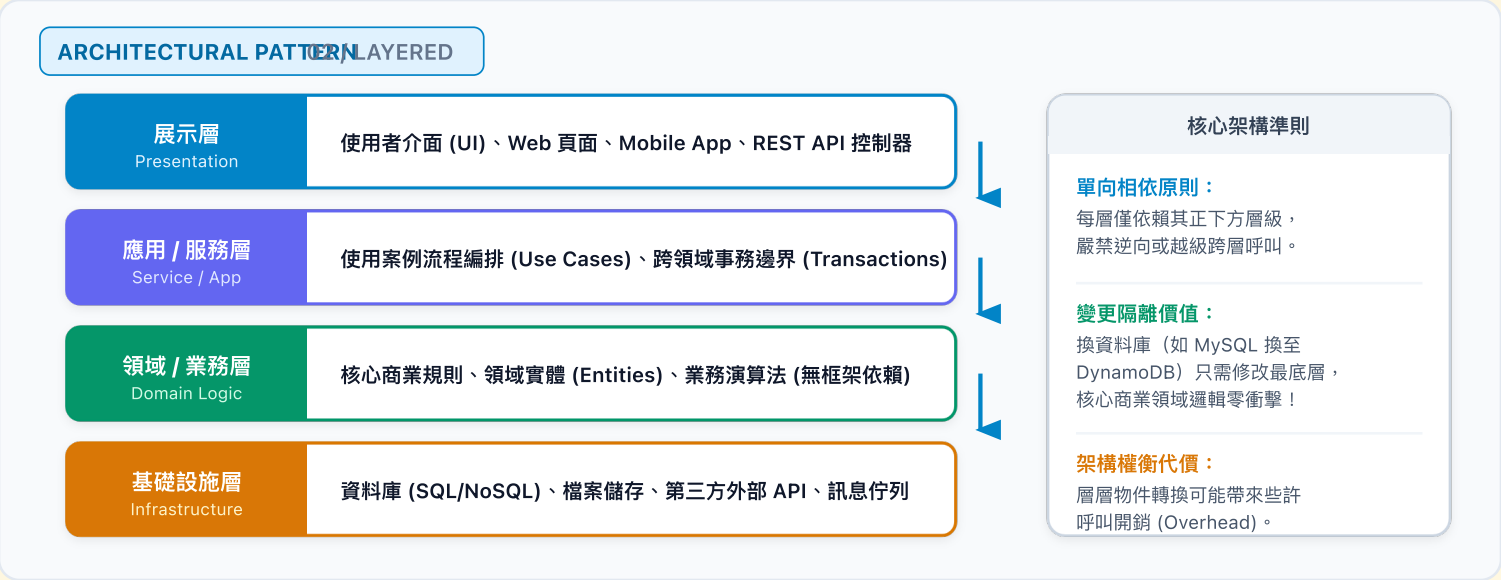
模式名稱	拓撲外觀特徵	核心運作機制	最適用場景	主要風險與承擔代價
1. MVC	三角循環迴圈（`C → M → V`）	徹底分離使用者介面與底層領域資料模型	具備豐富互動界面的 Web / 行動應用	對單純資料展示 (CRUD) 過於繁瑣複雜
2. 分層架構	垂直堆疊階層（`N → N-1`）	嚴格由上至下單向呼叫，跨層抽象隔離	企業級資訊系統、傳統關聯式資料庫應用	呼叫堆疊過深引發效能耗損（水槽反模式）
3. 儲存庫架構	星狀輻射樞紐架構	主動協同組件圍繞被動共用資料庫	編譯器、IDE 開發環境、多工資料分析系統	中央資料庫成為效能瓶頸與單一故障點 (SPOF)
4. 主從式架構	跨網路分散式（`C ↔ S`）	前端客戶端向集中式後端伺服器請求資源	Web 網站、檔案伺服器、集中管控服務	網路延遲風險；尖峰流量伺服器過載
5. 管線過濾器	線性單向資料流串接	獨立過濾器藉由單向管線串接串流數據	影音轉檔、大數據 ETL、編譯詞法分析	不適用於互動式 GUI；相鄰介面資料轉換開銷
6. 微服務架構	去中心化服務網狀拓撲	獨立部署自治微服務，各擁專屬私有 DB	超大型、團隊規模龐大、需極速迭代的雲平台	分散式系統複雜度：網路延遲、分散式交易

1. 模型-視圖-控制器模式 (Model-View-Controller, MVC)



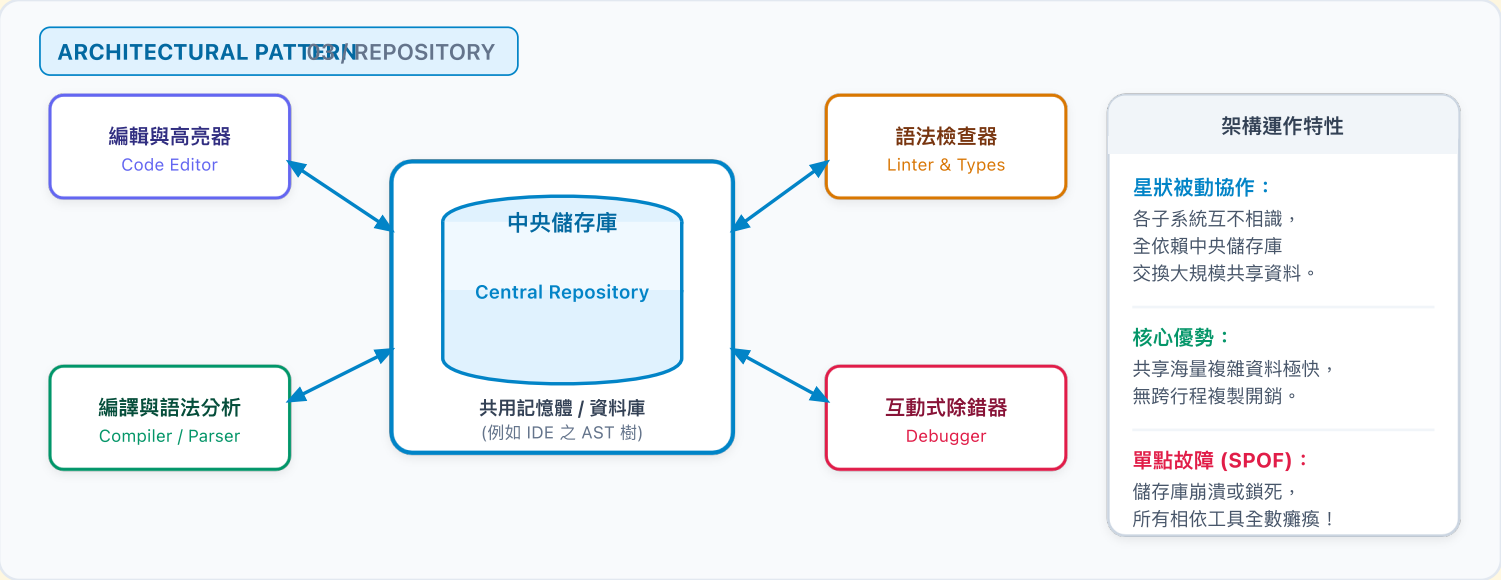
- **三大核心組件職責切分：**
  - **Model (模型)**：封裝核心領域商業資料、驗證規則與狀態，對視圖完全無知。
  - **View (視圖)**：負責將資料渲染成視覺介面呈現給使用者；監聽模型狀態變化。
  - **Controller (控制器)**：攔截使用者輸入與 HTTP 請求，轉譯並調度商業邏輯更新 Model。
- **工程優勢與取捨代價：**
  - **優勢**：同一資料模型可支援多種不同視圖（Web、App）；UI 前端與後端工程師可平行開發。
  - **代價**：在簡單介面中引入過多跳轉層級；若 View 直接碰 Model 易引發耦合破壞。

2. 分層架構模式 (Layered Architecture)



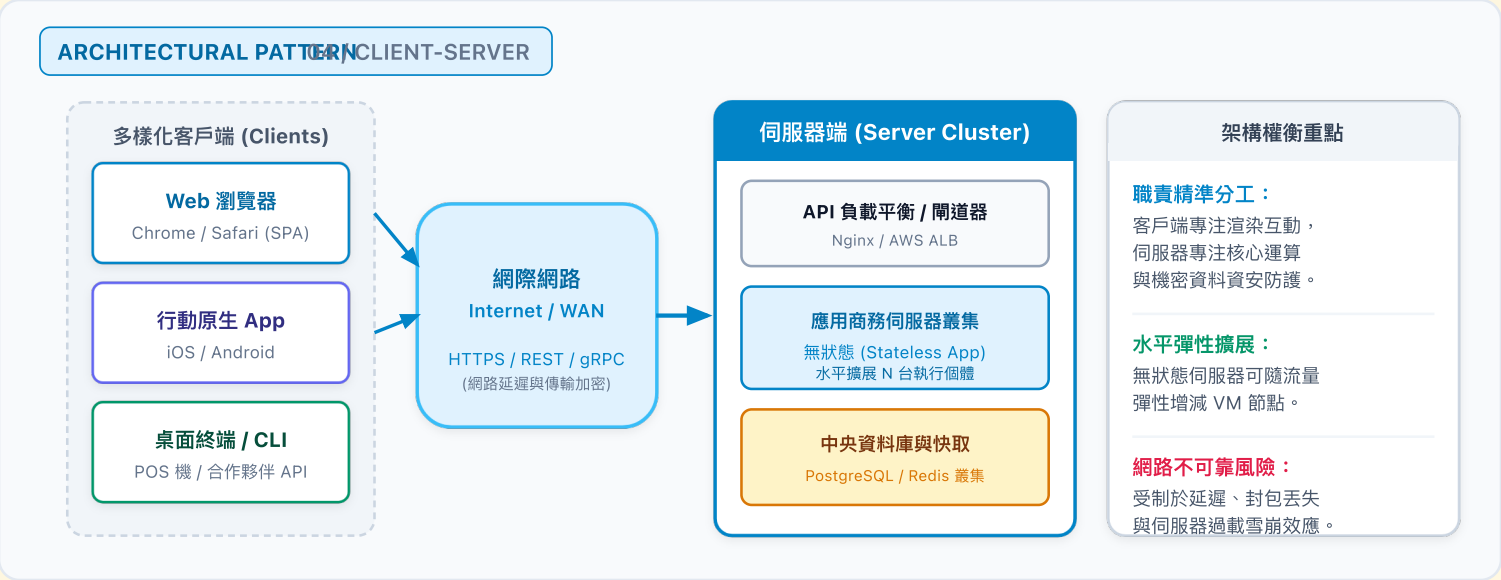
- **階層劃分與相依規則：**
  - 水平劃分為四層：**呈現層 (UI) → 商業邏輯層 (Business) → 資料存取層 (DAO) → 資料庫 (DB)**。
  - **嚴格分層原則 (Strict Layering)**：每一層只允許單向呼叫其緊鄰的下一層 (`Layer N → Layer N-1`)。
- **工程優勢與取捨代價：**
  - **優勢**：抽象層次清晰；模組替換性極佳（更換資料庫底層技術時，商業邏輯層完全無感）。
  - **代價**：**架構水槽反模式 (Architecture Sinkhole Anti-pattern)**（簡單查詢穿透多層卻不做事，徒增延遲）。

3. 儲存庫架構模式 (Repository Architecture)



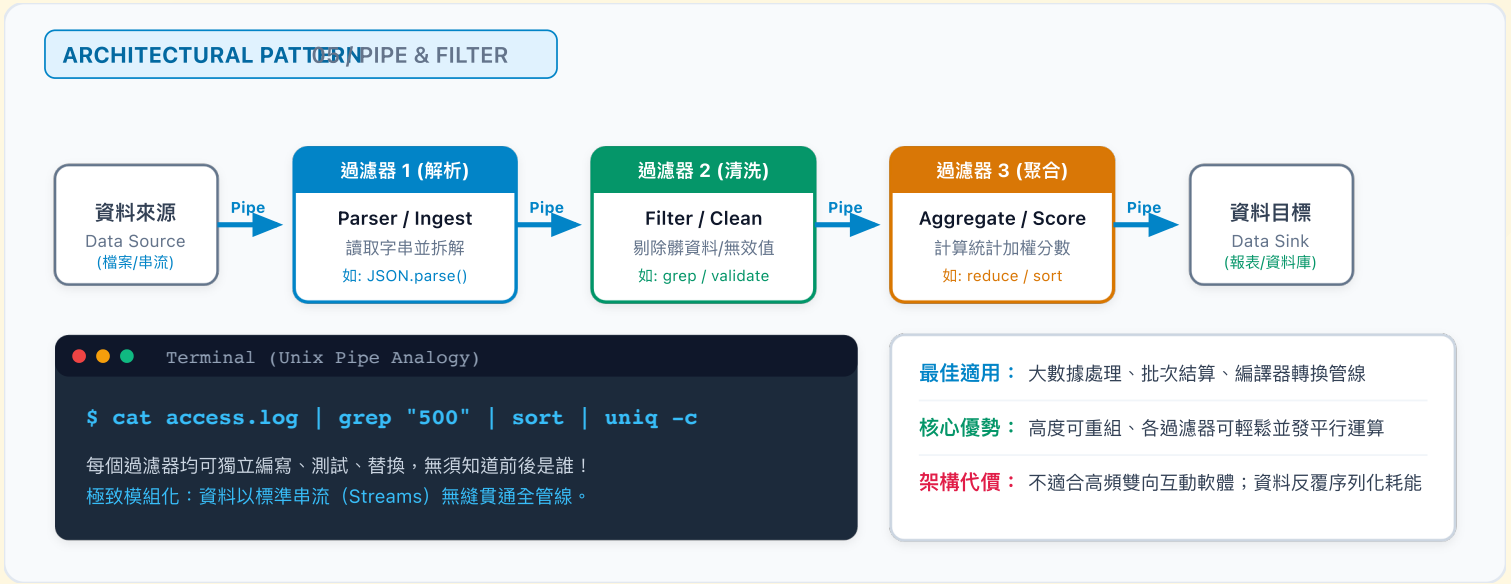
- **核心組件與拓撲特徵：**
  - **中央儲存庫 (Central Repository)**：維持系統所有持久化資料的被動共享儲存中心。
  - **獨立存取組件 (Active Subsystems)**：各自獨立運作的工具或子系統，僅透過中央庫進行資料交換。
- **工程優勢與取捨代價：**
  - **優勢**：極度適合共享海量複雜資料（如 IDE 的語法樹 AST、編譯器各階段 pass）；各工具間零直接相依。
  - **代價**：中央儲存庫極易成為全系統的效能瓶頸與**單一故障點 (SPOF)**；資料綱要變更牽一髮動全身。

4. 主從式架構模式 (Client-Server Architecture)



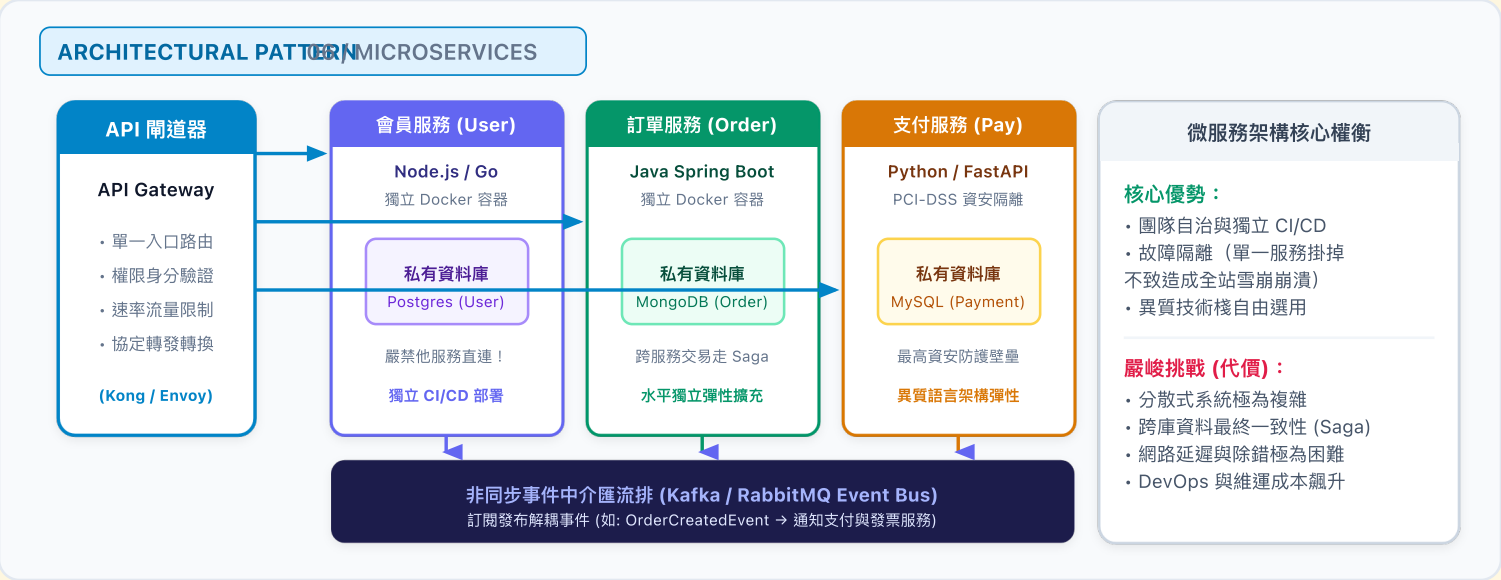
- **分散式節點分工：**
  - **伺服端 (Server)**：集中式服務提供者，提供資料儲存、安全運算與核心商業邏輯。
  - **客戶端 (Client)**：分散式消費者，負責向伺服端發送網路請求並呈現操作結果。
- **工程優勢與取捨代價：**
  - **優勢**：資料與商業權限高度集中治理；後端升級部署對使用者透明無感。
  - **代價**：受制於不可靠的網路延遲與中斷；尖峰流量易造成伺服器過載，需額外設計負載平衡與快取。

5. 管線與過濾器模式 (Pipe-and-Filter Architecture)



- **資料流串流機制：**
  - **過濾器 (Filters)**：獨立、無狀態的資料處理組件，讀取輸入串流，套用轉換後輸出至下一階段。
  - **管線 (Pipes)**：單向傳輸通道，負責緩衝並引導資料流從一個過濾器流向另一個過濾器。
- **工程優勢與取捨代價：**
  - **優勢**：極高可復用性（如 Unix 指令 `cat file | grep 'error' | sort`）；各階段極易並行或分散式加速。
  - **代價**：完全不適合互動式 GUI 系統；各組件間反覆序列化與反序列化文字資料帶來額外開銷。

6. 微服務架構模式 (Microservices Architecture, MSA)



- **去中心化自治服務：**
  - 將龐大單體系統分解為圍繞商業領域能力構建的**小型、獨立部署服務叢集**。
  - **一服務一資料庫 (Database-per-Service)**：各服務擁有專屬私有資料庫，嚴禁跨服務直接進行 SQL JOIN 查詢！
- **工程優勢與取捨代價：**
  - **優勢**：團隊自治與獨立部署發布步調；異質技術棧選擇；**卓越的故障衝擊半徑隔離 (Blast Radius)**。
  - **代價**：分散式系統噩夢：網路延遲、跨服務分散式交易 (Saga Pattern) 與最終一致性 (Eventual Consistency)。

單體架構 vs. 微服務架構：深度技術對比

比較維度	單體架構 (Monolithic Architecture)	微服務架構 (Microservices Architecture)
部署單元	單一巨大二進位執行檔 / WAR 包 / 容器	數十至數百個獨立運行的輕量微容器
資料儲存	單一共享全域關聯式資料庫 (Shared DB)	**各服務私有資料庫 (Database-per-Service)**
通訊成本	極速奈秒級程序內記憶體呼叫 (In-memory)	毫秒級網路呼叫 (RPC / REST / gRPC / 訊息佇列)
故障隔離	**極差**：模組 A 記憶體洩漏會導致全系統當機	**卓越**：金流服務崩潰，使用者依然能正常瀏覽商品
團隊擴展	龐大代碼衝突、部署協調成本隨人數激增	獨立特徵團隊，每週自主上線部署數十次
維運門檻	低門檻（單一 CI/CD 流水線、集中式日誌）	極高門檻（Kubernetes、分散式追蹤、服務網格）
適用時機	新創公司、MVP 概念驗證、小型團隊 (<15人)	組織規模龐大 (>50人)、業務領域極度複雜之企業

軟體架構直接決定非功能品質屬性

- **品質屬性無法在軟體完工後靠補丁外掛：**
  - 你無法建造一個高度緊耦合的單體系統，然後指望靠寫幾個單元測試就讓它具備百萬並行能力。
- **架構決策如何實質主宰品質屬性：**
  - ⚡ **效能 (Performance)**：由跨網路呼叫跳轉數、記憶體快取層與資料庫分割策略直接決定。
  - 🛡️ **安全性 (Security)**：由零信任網路隔離、API 閘道防護與機密資料庫階層切分嚴格把關。
  - 🦺 **安全性/防護力 (Safety)**：由關鍵控制核心與非關鍵模組之物理隔離所保障（如汽車煞車 vs. 影音娛樂系統）。
  - 🔄 **可用性與容錯 (Availability)**：由冗餘備援節點、健康檢查、斷路器 (Circuit Breaker) 與自動容錯轉移所構築。
  - 🔧 **可維護性 (Maintainability)**：由系統是否嚴格落實高凝聚與低耦合邊界所主宰。

架構權衡分析 (Architecture Trade-off Analysis) 與 ATAM

- **不可避免的衝突矩陣 (The Inevitable Conflict Matrix)：**
  - 軟體架構中**不存在完美解法，只有取捨與妥協 (Trade-offs)**：
    - **效能 vs. 安全性**：高強度加密演算法與多層身分驗證必然增加連線延遲。
    - **可用性 vs. 一致性 (CAP 定理)**：分散式系統中，追求絕對一致性必然犧牲高可用響應。
    - **模組化 vs. 執行效能**：極致解耦的多層呼叫，必然帶來物件序列化與堆疊開銷。
- **ATAM (架構權衡分析法 / Architecture Trade-off Analysis Method)：**
  - 由卡內基美隆 SEI 研發之業界標準架構評估方法論，旨在編寫代碼前預先發掘潛在架構風險：
  - **ATAM 核心概念：**
    - **敏感點 (Sensitivity Points)**：對單一品質屬性具有決定性影響的關鍵架構決策。
    - **權衡點 (Trade-off Points)**：同時影響多個互斥品質屬性的兩難決策點（如加密提升安全卻壓低效能）。

觀念檢核測驗 2 (CCQ 2)

在將單體架構 (Monolithic) 與微服務架構 (Microservices) 進行對比時，微服務之所以能夠提供卓越的**故障隔離能力 (Fault Isolation / 縮小衝擊半徑)**，其最核心的結構性原因為何？

- **A.** 微服務將全企業所有商業資料集中儲存於單一全域關聯式資料庫中。
- **B.** 每個微服務皆獨立運行於專屬的獨立行程 (Process) 與私有資料庫中，防止骨牌式連鎖崩潰。
- **C.** 微服務架構完全消除了跨網路通訊的所有封包傳輸延遲與開銷。
- **D.** 微服務架構大幅精簡了專案所需的軟體工程師與系統維運人力。



5.4 軟體架構推導與塑模表現：超越 Pattern

> *「架構是那些你希望一開始就能做對的重要決策。」*
> *— Ralph Johnson (Design Patterns 作者)*

架構如何推導？超越 Pattern 挑選的 5 步決策流程

- **架構絕非「挑選 Pattern 的購物車」：**
  - 架構模式只是積木與範本；架構設計的本質是一套嚴謹的**推導、分解與權衡流程**：
- **架構推導五大核心步驟：**
  - **1. 提煉 ASR (關鍵架構需求 / Architecturally Significant Requirements)**：
    - 並非所有需求都影響架構！聚焦於具決定性影響的非功能品質屬性（如 99.99% 可用性、<100ms 延遲）與核心使用案例。
  - **2. 系統分解與邊界劃分 (Decomposition)**：
    - 依據業務領域能力（DDD 限界上下文 Bounded Contexts）或技術層次切分子系統。
  - **3. 組合與合成架構模式 (Pattern Synthesis)**：
    - 現實系統皆為複合架構：外層微服務 Mesh、中間事件驅動管線、內部則是分層 MVC！
  - **4. 定義介面合約與通訊協定 (Protocols & Interfaces)**：
    - 同步 RESTful / gRPC vs. 異步 Kafka / RabbitMQ 事件流。
  - **5. 評估驗證與沉澱決策 (Evaluate & Document)**：
    - 透過 ATAM 評估極端情境，並以 ADR 記錄決策脈絡。

如何畫架構圖？傳統 UML 在實務中的定位與陷阱

- **「巨大類別圖陷阱」 (學生最常見的嚴重誤區)：**
  - 學生常畫出一張擠滿 50 個類別的 UML Class Diagram，宣稱那是「系統架構圖」。
  - **致命盲點：** 類別圖只呈現編譯期的物件關聯；它**完全掩蓋了網路通訊、程序隔離、伺服器節點與雲端部屬邊界**！
- **傳統 UML 在巨觀架構中的正確用法：**
  - **1. 組件圖 (Component Diagram)**：
    - 呈現高階軟體組件、提供的介面 (`--○` 棒棒糖符號) 與相依的介面 (`--(` 插座符號)。
  - **2. 套件圖 (Package Diagram)**：
    - 呈現命名空間分群、子系統模組邊界與套件相依方向。
  - **3. 部屬圖 (Deployment Diagram)**：
    - 呈現實體硬體節點、虛擬機器、容器、執行環境與網路協定 (HTTPS, TCP/IP, JDBC)。

多維度架構視圖：Kruchten 的 4+1 架構視圖模型

- **Philippe Kruchten (1995) 經典洞察：「沒有單一圖表能滿足所有利益關係人！」**
- **4+1 多維度架構模型：**
  - **1. 邏輯視圖 (Logical View / 終端使用者與領域功能)**：
    - 系統的類別結構、領域概念與物件封裝，滿足系統的功能性需求。
  - **2. 程序視圖 (Process View / 整合工程師與效能調校)**：
    - 系統的並行運算 (Concurrency)、執行緒、非同步訊息佇列、處理行程間通訊 (IPC) 與回應效能。
  - **3. 開發視圖 (Development View / 軟體工程師與專案管理)**：
    - 原始碼套件組織、第三方函式庫相依、Maven/Gradle 建置模組結構。
  - **4. 實體視圖 (Physical View / 系統維運與 DevOps)**：
    - 實體伺服器、Docker 容器叢集、網路拓撲、防火牆邊界與部署通訊協定。
  - **+1. 核心場景 (Scenarios / 驅動與驗證核心)**：
    - 貫穿並驗證前述四個視圖的核心業務使用案例 (Use Cases)。

現代架構視覺化主流：C4 Model

- **Simon Brown 創立之現代、直觀且開發者友善的架構表達標準：**
- **「軟體架構的 Google Maps」——四層層次化縮放邏輯：**
  - 🗺️ **Level 1: System Context (系統情境圖 / 全景空照圖)**：
    - 誰在使用系統？系統與哪些外部系統整合（如第三方金流 Stripe、Google SSO）？
  - 🏙️ **Level 2: Container Diagram (容器圖 / 城市街區圖)**：
    - 包含哪些可獨立部署的單元？（SPA 前端、手機 App、API 後端、關聯資料庫、Redis 快取）與其通訊協定（JSON/HTTPS, gRPC）。
  - 🏛️ **Level 3: Component Diagram (組件圖 / 建築平面圖)**：
    - 放大單一 Container 內部：包含哪些核心架構組件（如 AuthController, PaymentService, UserRepository）？
  - 🚪 **Level 4: Code Diagram (程式碼圖 / 室內裝潢圖)**：
    - 類別圖特寫（通常為選填；只針對極度複雜的演算法模組繪製）。

沉澱架構決策：架構決策記錄 (Architecture Decision Records, ADR)

- **為什麼必須撰寫 ADR：**
  - 程式碼忠實記錄了「*怎麼做 (How)*」，但程式碼從不解釋「*為什麼這麼做 (Why)*」。
  - 缺乏 ADR 會引發**「架構考古學 (Architectural Archaeology)」**——新進工程師花費數月苦思當年為何選擇這套非主流技術。
- **標準 ADR 結構 (Michael Nygard 輕量模板)：**
  - **1. 標題 (Title)**：編號與明確決策（例如：`ADR-003: 採用 PostgreSQL 作為交易持久化資料庫`）。
  - **2. 狀態 (Status)**：`提議 (Proposed)` → `接受 (Accepted)` → `廢棄 (Deprecated)` → `被取代 (Superseded)`。
  - **3. 背景脈絡 (Context)**：面臨的商業問題、技術約束與相互抗衡的力量。
  - **4. 決策內容 (Decision)**：採取的具體架構方案與選型依據。
  - **5. 後續影響 (Consequences)**：帶來的好處，以及**明確承擔的負面代價與權衡**！

觀念檢核測驗 3 (CCQ 3)

在 Philippe Kruchten 提出的 4+1 架構視圖模型 (4+1 View Model) 中，哪一個視圖專門負責描繪軟體組件如何被部屬至實體伺服器硬體、雲端容器叢集與底層網路拓撲上？



- **A.** 邏輯視圖 (Logical View)
- **B.** 程序視圖 (Process View)
- **C.** 開發視圖 (Development View)
- **D.** 實體視圖 / 部屬視圖 (Physical View / Deployment View)

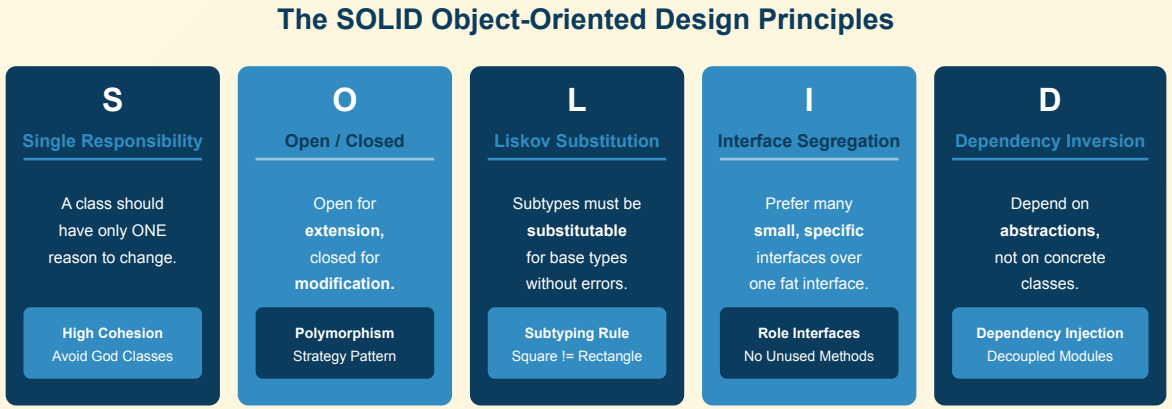
5.5 SOLID 物件導向設計原則

> *"優良的程式碼總是看起來像是出自一位真正關心它的人之手。"*
> *— Robert C. Martin (Uncle Bob)*

從巨觀架構邁向微觀詳細設計 (From Architecture to Detailed Design)

- **架構藍圖與原始代碼的深度連結：**
  - 巨觀架構建立了系統骨架與子系統邊界。
  - 微觀詳細設計則以健全、易讀的類別與方法填滿該骨架。
- **防範「微觀腐敗 (Micro-Decay)」危機：**
  - 即便採用最先進的微服務或分層架構，若單一服務內部的類別退化為高耦合的義大利麵代碼，整體系統依然會崩塌！
- **SOLID 五大原則全稱 (Robert C. Martin 提煉)：**
  - **S**：單一職責原則 (Single Responsibility Principle, SRP)
  - **O**：開放封閉原則 (Open / Closed Principle, OCP)
  - **L**：里氏替換原則 (Liskov Substitution Principle, LSP)
  - **I**：介面隔離原則 (Interface Segregation Principle, ISP)
  - **D**：相依反轉原則 (Dependency Inversion Principle, DIP)

概覽：SOLID 五大設計原則



- **SOLID 原則核心主張：**
  - **SRP**：一個類別應當僅有一個引起它改變的原因。
  - **OCP**：軟體實體應當對擴充開放，對修改封閉。
  - **LSP**：子型態必須能夠無痛替換其基礎型態而不影響程式正確性。
  - **ISP**：用戶端不應被迫依賴其所不需要的方法。
  - **DIP**：高階模組不應依賴低階模組；兩者皆應依賴於抽象介面。

S – 單一職責原則 (Single Responsibility Principle, SRP)

- **核心原則：**

> *一個類別應該有且僅有一個引起它改變的原因 (Reason to Change)。*

- **為什麼 SRP 至關重要：**
  - 當一個類別承擔多重職責時，來自不同業務利益關係人的需求變更會混雜在一起，導致非預期的迴歸錯誤。

```
// ❌ 違反 SRP：承擔多重責任的「上帝類別」，擁有多個改變理由
class UserInvoiceReport {
    public void calculateTotal()  { /* 財務部規則變更 */ }
    public void renderHtmlPage()  { /* 行銷部 UI 樣式變更 */ }
    public void saveToDatabase()  { /* DBA 資料表欄位變更 */ }
}
```

```
// ✅ 遵循 SRP：各司其職，每個類別只有單一改變理由
class InvoiceCalculator { public BigDecimal calculateTotal(Invoice inv) { ... } }
class InvoiceHtmlRenderer { public String render(Invoice inv) { ... } }
class InvoiceRepository { public void save(Invoice inv) { ... } }
```

O – 開放封閉原則 (Open / Closed Principle, OCP)

- **核心原則 (Bertrand Meyer 提出)：**

> *軟體實體（類別、模組、函式）應當對擴充開放 (Open for extension)，但對修改封閉 (Closed for modification)。*
>
> ”

- **如何達成 OCP：**
  - 依賴抽象介面；當需要加入新行為時，透過「新增新的類別」來擴充，**絕不開刀修改已通過測試的既有程式碼！**

```
// ❌ 違反 OCP：每新增一種通知管道，就必須打開既有類別硬改 if/else！
if (type.equals("EMAIL")) { sendEmail(); }
else if (type.equals("SMS")) { sendSms(); }
else if (type.equals("DISCORD")) { sendDiscord(); } // 風險極高的修改！
```

```
// ✅ 遵循 OCP：藉由多型介面，以新增實作類別達成無痛擴充
interface NotificationChannel { void send(String message); }
class EmailChannel implements NotificationChannel { ... }
class DiscordChannel implements NotificationChannel { ... } // 核心引擎零修改！
```

L – 里氏替換原則 (Liskov Substitution Principle, LSP)

- **核心原則 (圖靈獎得主 Barbara Liskov 提出)：**

> *子型態必須能夠無痛替換其基礎型態，而不會破壞程式的任何預期正確性。*

- **經典反例：正方形繼承長方形 (The Square-Rectangle Problem)：**
  - 數學上正方形是長方形；但在物件導向中，覆寫 `setWidth(w)` 連帶修改 `height` 會破壞呼叫端的契約期待！

```
// ❌ 違反 LSP：覆寫引發非預期副作用，破壞基礎契約
class Rectangle {
    protected int w, h;
    public void setWidth(int w) { this.w = w; }
    public void setHeight(int h) { this.h = h; }
    public int getArea() { return w * h; }
}
class Square extends Rectangle {
    @Override public void setWidth(int w) { this.w = this.h = w; } // 破壞了調用端期待！
}
```


- **工程守則：** 子類別絕不可放寬前置條件 (Preconditions)，亦不可縮減後置條件

I – 介面隔離原則 (Interface Segregation Principle, ISP)

- **核心原則：**

> *用戶端不應該被迫依賴其所不需要的方法。*
>
> ”

- **核心主張：** 寧可建立多個**精巧專一的角色介面 (Role Interfaces)**，絕不要打造包山包海的「肥大介面 (Fat Interface)」。

```
// ❌ 違反 ISP：肥大介面強迫實作者填入無意義的虛構實作
interface MultiFunctionPrinter {
    void print();
    void scan();
    void fax();
}
class SimpleInkjetPrinter implements MultiFunctionPrinter {
    public void print() { /* 正常列印 */ }
    public void scan()  { throw new UnsupportedOperationException(); } // 遭受污染！
    public void fax()   { throw new UnsupportedOperationException(); } // 遭受污染！
}
```

```
// ✅ 遵循 ISP：精確切分角色介面，各取所需
```

D – 相依反轉原則 (Dependency Inversion Principle, DIP)

- **核心原則：**

> 1. *高階模組不應依賴低階模組；兩者皆應依賴於抽象介面。*
> 2. *抽象介面不應依賴於細節實作；細節實作必須依賴於抽象介面。*
>
> ”

```
// ❌ 違反 DIP：高階商業邏輯與特定底層 MySQL 資料庫死死綁定
class OrderService {
    private MySQLDatabase db = new MySQLDatabase(); // 直接 'new' 出具體實作！
}
```

```
// ✅ 遵循 DIP：依賴注入抽象介面 (Dependency Injection)
interface Database { void save(Order o); }
class MySQLDatabase implements Database { ... }
class MockDatabase implements Database { ... } // 單元測試秒跑記憶體假物件！

class OrderService {
    private final Database db;
    public OrderService(Database db) { this.db = db; } // 由外部注入抽象！
}
```

觀念檢核測驗 4 (CCQ 4)

哪一個 SOLID 物件導向設計原則明確主張：當需要為系統引入新功能時，應當透過「新增新的類別」來擴充，而非直接修改既有已通過測試的原始碼？

- **A.** 單一職責原則 (SRP)
- **B.** 開放封閉原則 (OCP)
- **C.** 介面隔離原則 (ISP)
- **D.** 里氏替換原則 (LSP)



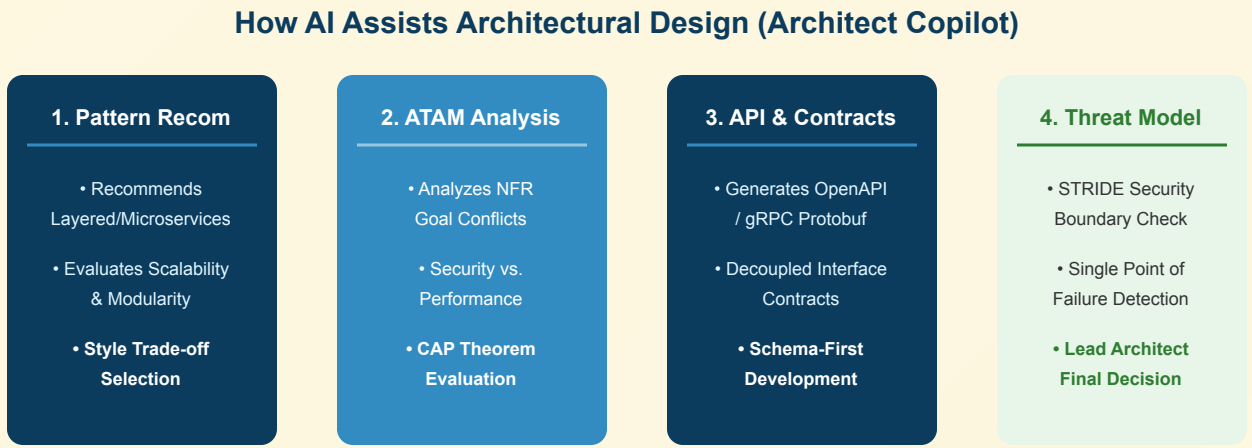
5.6 AI 在架構與軟體設計中的賦能與盲點

> *"AI 不會取代架構師，但善用 AI 的架構師將會取代不用 AI 的人。"*

AI 在架構與設計中的角色：副駕駛典範 (The Copilot Paradigm)

- **軟體架構師工具庫的典範轉移：**
  - **傳統工作流**：純手工白板繪圖、耗時冗長的人工作業評審、步履維艱的手工重構。
  - **AI 賦能工作流**：對話式架構權衡探索、自動化架構異味掃描、提示詞驅動介面生成。
- **「是副駕駛 (Copilot)，不是自動駕駛 (Autopilot)」鐵律：**
  - AI 擅長於**發散生成多元備選方案、快速草擬文檔與列舉極端情境**。
  - 人類架構師必須提供**組織團隊脈絡、雲端維運預算、法規資安邊界與最終責任承擔**。

AI 在軟體設計中的 4 大核心落地應用



- **1. 架構權衡探索與極端壓力演練 (ATAM 輔助)：**
  - 提示 LLM 扮演紅隊進行破壞性思考：「若 Redis 在分區故障期間崩潰，我們的支付管線會如何表現？」
- **2. 自動化代碼異味與反模式掃描 (Code Smell Detection)：**
  - 在 PR 合併前自動偵測上帝類別、功能依戀 (Feature Envy)、循環相依與違背 SRP 之處。
- **3. 設計模式重構建議 (Design Pattern Refactoring)：**
  - 將巢狀僵化的 `switch/case` 自動重構為多型策略模式 (Strategy) 或工廠模式 (Factory)。
- **4. 架構文檔與決策記錄生成 (ADR Drafting)：**
  - 自動草擬標準架構決策記錄 (ADR) 與 OpenAPI / gRPC 介面合約草案。

人機協同防護：防範 AI 架構盲點 (Architectural Blind Spots)

- **盲目採納 AI 產生的三大致命風險：**
  - **過度複雜化 (Resume-Driven Architecture)**：AI 經常過度工程化，為 3 個人兩週就能搞定的內部工具推薦 Kubernetes、微服務與 Kafka！
  - **組織脈絡盲目 (Context Blindness)**：AI 完全不知道你們團隊無人會除錯分散式競態條件，也無視你們的雲端預算上限。
  - **膚淺表面重構 (Superficial Refactoring)**：僅替換變數名稱，卻未根本解決資料耦合與架構瓶頸。
- **系統架構的至高金科玉律：**

> ***「AI 提案得失，架構師裁奪決策！」***
> *人類工程師必須依據現實組織環境做裁量，並對系統的長期健全負起終極責任。*

5.7 核心概念統整與參考文獻

> *"簡單是可靠性的絕對前提。"*
> *— Edsger W. Dijkstra (圖靈獎得主)*

本章核心精華統整矩陣 (Curriculum Synthesis Matrix)

課程維度	核心本質概念	最關鍵架構啟發 Heuristic
5.1 設計導論	四大設計層次	儘早捕捉架構盲點；品質屬性是設計出來的，不是測出來的。
5.2 通用原則	高凝聚與低耦合	**極大化凝聚度，極小化耦合度！** 縮小變更衝擊半徑。
5.3 架構塑模	4+1 視圖與 C4 Model	超越模式挑選：以 ASR 需求驅動推導，以 ADR 記錄決策脈絡。
5.4 經典模式	六大架構模式	軟體工程沒有完美解答，只有嚴謹評估的權衡 (ATAM)。
5.5 詳細設計	SOLID 五大原則	依賴抽象而非細節 (DIP)；對擴充開放，對修改封閉 (OCP)。
5.6 AI 賦能	副駕駛典範	AI 評估代碼與得失，人類架構師把關組織現實脈絡！

經典學術與業界參考文獻 (References)

- **軟體架構經典教科書與權威文獻：**
  - Len Bass, Paul Clements, Rick Kazman (SEI): *Software Architecture in Practice*, 4th Edition, Addison-Wesley.
  - Philippe Kruchten: "*Architectural Blueprints—The 4+1 View Model of Software Architecture*", IEEE Software, 1995.
  - Simon Brown: *Software Architecture for Developers (The C4 Model)*, Leanpub.
- **經典設計原則與設計模式著作：**
  - Robert C. Martin (Uncle Bob): *Clean Architecture: A Craftsman's Guide to Software Structure and Design*, Prentice Hall.
  - Erich Gamma, Richard Helm, Ralph Johnson, John Vlissides: *Design Patterns: Elements of Reusable Object-Oriented Software (GoF)*, Addison-Wesley.
  - Martin Fowler: *Patterns of Enterprise Application Architecture (P of EAA)*, Addison-Wesley.
  - Michael Nygard: *Documenting Architecture Decisions (ADR)*, 2011.

感謝聆聽，歡迎提問與深入交流！

Q & A · 共同討論

薛念林 教授
資訊工程學系
逢甲大學